

Jonathan Ekman, Brenden Moran

EECE 490: Capstone

Advisor: Dr. Reza Khani

Chiplet Based Architecture Prototype

Test #1: Accelerator Logic

A main part of this project is the ability for others to follow the same procedure as us to test and verify their own SoC (system on a chip) to gather evidence of it improving the computational cost of the system. To prove our system works we first need to develop our own accelerator chip to display how others could do the same. This subsystem that is under test will be the accelerator logic itself, this is to prove that our developed accelerator can multiply 2 matrices together to gain an output matrix.

The way this subsystem was tested is by developing a testbench in system verilog to verify the workings of our accelerator. The testbench specifically tests how the matrix multiplication works with 2 matrices of the same size (5x5) and then another test to verify a larger matrix of size (10x10). The reason for the multiplication being done by matrices of the same size is because the test case we chose for number identification requires 8x8 matrices to be multiplied together, therefore the 5x5 and 10x10 matrix multiplication tests will verify that for our test case, the accelerator will work.

The only required equipment for the test is the Vivado design suite to run the accelerator system and testbench.

- To Download Vivado: <https://www.xilinx.com/support/download.html>

To test the subsystem simply add both the accelerator system verilog file to the design sources portion in Vivado, and import the system verilog testbench file to the simulation sources of the project.

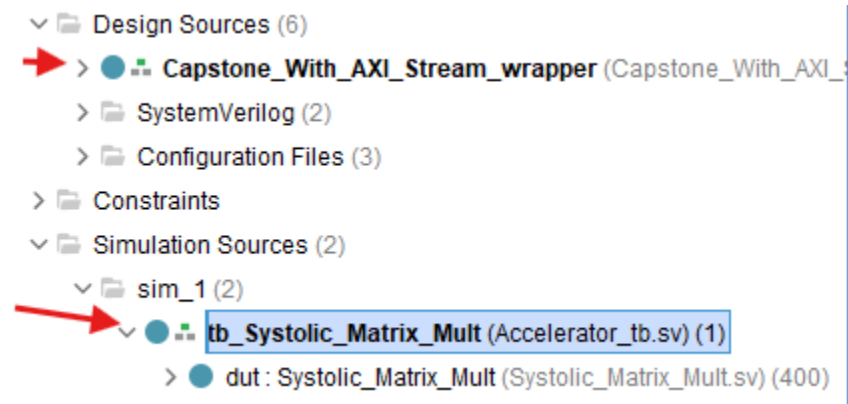


Fig. 1: Location of design and simulation sources in Vivado.

After both files are imported to the project, set both as the top level in their hierarchy. After which press the “run behavioral simulation” on the left of the Vivado design suite. This will run your test and output any messages to the console.

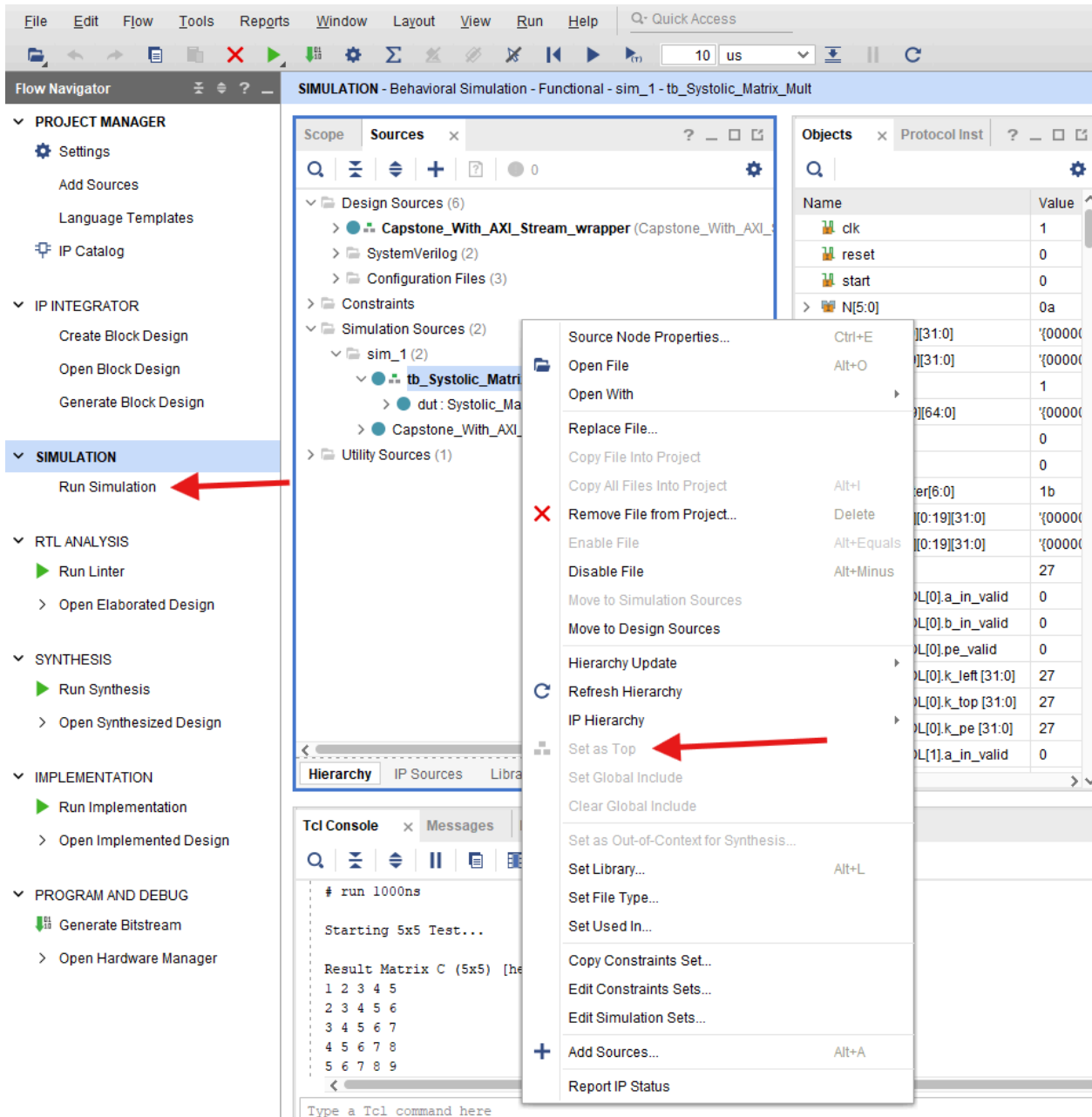


Fig. 2: Where the set as top hierarchy and run simulation buttons are in Vivado.

For our test we have the testbench output messages to the console that tell us whether or not the matrix multiplication ran correctly. We expect both of the matrices to output properly and then confirm that it is the correct value by printing a “passed” message at the end.

```

Starting 5x5 Test...

Result Matrix C (5x5) [hex]:
1 2 3 4 5
2 3 4 5 6
3 4 5 6 7
4 5 6 7 8
5 6 7 8 9
PASS: 5x5

Starting 10x10 Test...

Result Matrix C (10x10) [hex]:
1 2 3 4 5 6 7 8 9 a
2 3 4 5 6 7 8 9 a b
3 4 5 6 7 8 9 a b c
4 5 6 7 8 9 a b c d
5 6 7 8 9 a b c d e
6 7 8 9 a b c d e f
7 8 9 a b c d e f 10
8 9 a b c d e f 10 11
9 a b c d e f 10 11 12
a b c d e f 10 11 12 13
PASS: 10x10

Simulation Complete

```

Fig. 3: Test results printed from the test bench in Vivado design suite.

As seen in figure 3 above, both tests ran properly and the matrix multiplication gave us correct values for our output. With this portion confirmed we can then move onto creating our own custom IP with AXI lite and AXI stream communications that has this matrix multiplication embedded within it. Ensuring this module works on its own, allows us to not worry about it when debugging the system with all communications connected because we have already confirmed that the system works correctly.

Test #2: Block Diagram Verification

The subsystem under test here is the block diagram connecting all components together. This test should be run whenever the block diagram is changed in the Vivado design suite. The reason for this is because this validation shows whether or not Vivado itself has found an issue in your design. In all cases of this project, running this test has been integral to understanding how each component communicates with one another. This test also must validate correctly or else communication between components cannot be confirmed.

The test itself is very straightforward. It is a Vivado built in test where it validates the block diagram and produces error messages based on what components are incorrectly connected.

The required equipment is just the Vivado Design Suite.

We expect that when everything runs correctly there should be no error messages at the output and the validation should have run successfully. To run the test simply press the “validate design” button at the top as seen in figure 4.

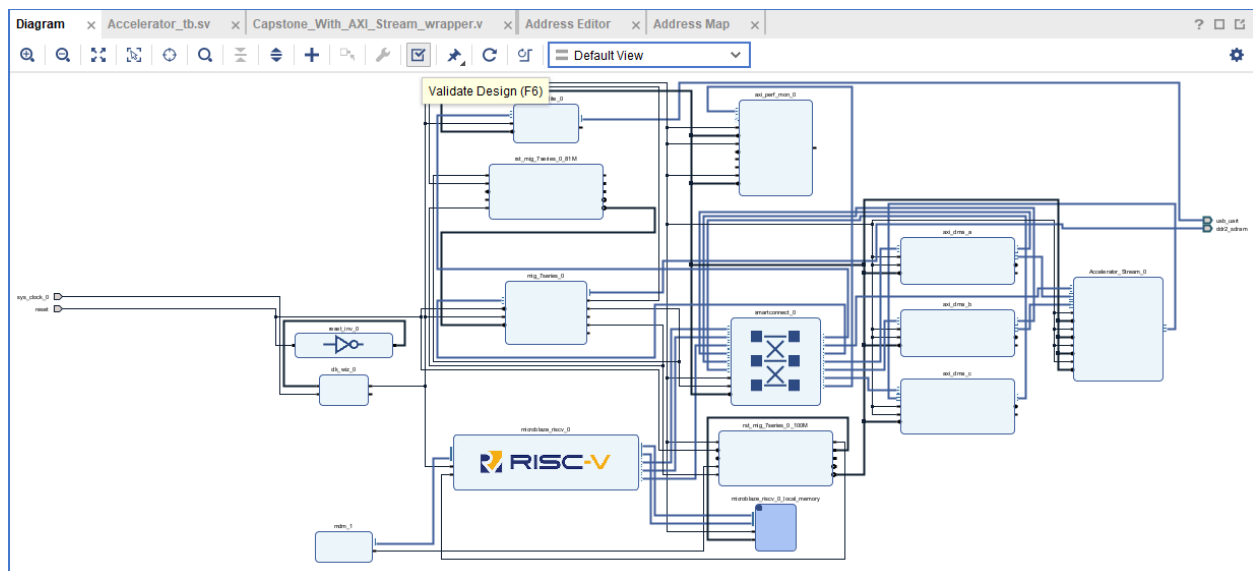


Fig. 4: Validate block diagram button.

Our design validates correctly proving that all communication should be connected properly and our system should be able to run just from an architectural view. Below in figure 5 shows the output from this test.

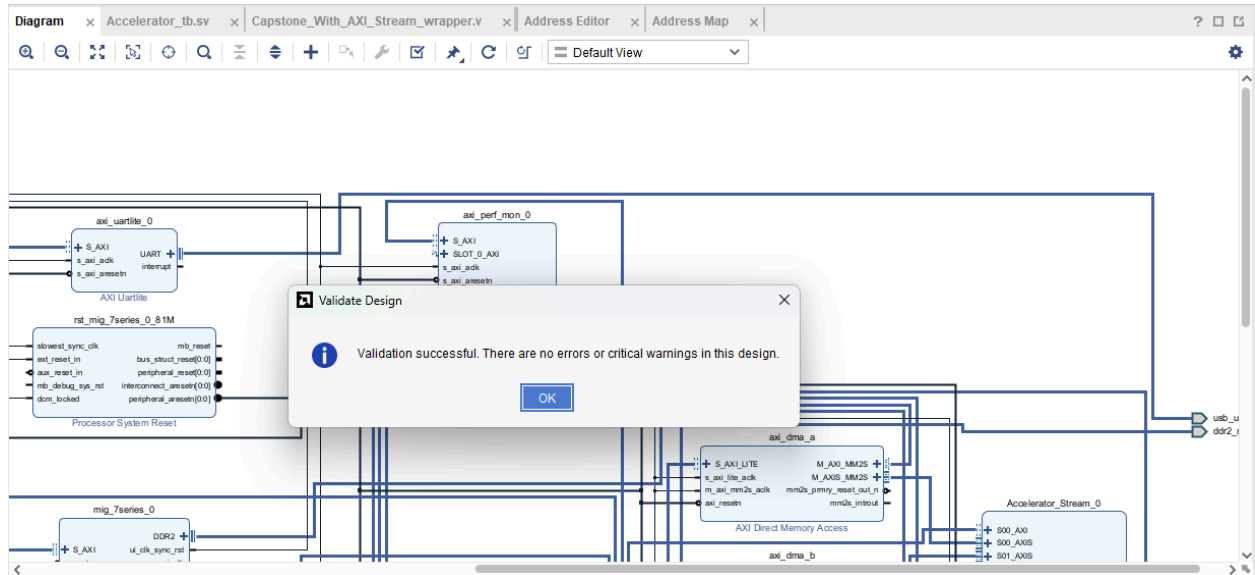


Fig 5: Output from successful block diagram validation test.

Test #3: AXI Lite/Stream Peripheral Communication

The AXI Lite and Stream Peripheral Communication subsystem is very important to our design. This has to do with the communication to our custom IP. The way it is embedded within our design is that our custom matrix multiplication module cannot communicate on its own with the rest of our system, because of this we need to create an AXI peripheral wrapper which is in charge of communication between our custom module and the rest of the system. This is where AXI lite and AXI stream come in. The AXI Lite peripheral is great for register mapping, where the main CPU can send single 32-bit integers to each AXI lite register. This works well for simple control bits which we use in the matrix multiplication module. As for transferring the entire matrix to the module, using AXI lite registers would be cumbersome. Because of this we

opted to use AXI stream, which allows the DDR to stream the input matrices to our custom IP through DMAs, and then allowing our IP to stream the output matrix back to the DDR.

The test itself is relatively easy. Both AXI lite and stream communication need to be working properly for the system to multiply 2 matrices. We can use this to our advantage by making a C code in Vitis which multiplies two matrices using our new custom IP, which will be downloaded to our system. If all communication runs correctly we should see the two matrices being multiplied together and receiving our correct output.

The equipment needed for this test is the Vitis IDE which is used to create a platform and application to run on our design and Putty to see the output in a console.

- <https://putty.org/index.html>

The code uses the base addresses found in xparameters.h file to communicate with the DMAs as well as the matrix multiplier IP itself. This header file is autogenerated by Vitis when a platform and application are built. Our C file can be found in the “test_3” folder.

For our test to run properly we added a software matrix multiplication which will give us the correct value of the multiplication also done in hardware. We expect both of these to be equal, but if not we know something in communication did not run properly specifically with AXI stream. If AXI lite had any errors the system would never print any output because the control bits would never reach the module itself. We also run a back to back test to ensure that no values are corrupted from the previous multiplication computed.

```

=====
AXI-Stream Accelerator Repeated-Run Test
=====
DMA_A initialized (base=0x41E00000)
DMA_B initialized (base=0x41E10000)
DMA_C initialized (base=0x41E20000)

--- AXI-Lite communication test ---
REG_N      = 3
REG_STATUS = 0x00000000
AXI-Lite communication OK.

=====
Test 1: First-term double-count check
=====
SW Expected C =
    11      12      13
    0        0        0
    0        0        0
Final status = 0x00000001
HW C =
    11      12      13
    0        0        0
    0        0        0
PASS: Hardware result matches software reference.

=====
Test 2: Second multiply after first
=====
SW Expected C =
    45      62      39
    80      96      53
    85      76      46
Final status = 0x00000001
HW C =
    45      62      39
    80      96      53
    85      76      46
PASS: Hardware result matches software reference.

Both back-to-back tests completed.

```

Fig. 6: Output of AXI Stream and Lite C code test.

From our output above we can confirm that all communications work and no values are corrupted internally from the back to back test. AXI Lite works because the test ran and was not stopped or corrupted in the registers. AXI Stream ran correctly because all matrix values are correct which means no bad values were passed into the module. Finally the back-to-back test

ran which proves no values are inherently corrupted when running multiple matrix multiply computations over and over again.

Test #4: Measurement Metrics

This test confirms the measurements that can be obtained from the system, either in the CPU from on board clock/counters or through APMs for direct AXI bus measurements. This test confirms that for future users they will be able to see how much faster their custom accelerators could make the system while also giving us metrics for how well our system accelerates computation.

The way this test works is by using the MicroBlaze V to keep track of CPU cycles needed to run certain algorithms. The algorithm we chose is relatively common in machine learning where given an input matrix which represents a number, this will be compared to multiple reference matrices which when multiplied together with other specific math can deduce which number that matrix represents. This is a simple algorithm that could be switched out for any other algorithm that uses matrix multiplication; specifically you would just have to alter the code slightly to be able to communicate with the accelerator IP.

This test does not directly have a pass/fail criteria as we are more interested in it running properly than actual performance metrics which was already proven in the previous tests, but it would be very nice if our accelerator showed latency improvements. The current algorithm only used 8x8 matrices so the software may be faster or much similar in latency than it would be if the matrices were much larger.

The needed equipment for this test is Vitis IDE, Putty for output terminal, and the test code which can be found in the “test_4” folder.

Both test outputs can be seen below through Putty, where figure 7 is the hardware only test and figure 8 is the software only test.

```
=====
Hardware-Only Latency Test
Tests only: digit 3 and digit 6
=====
DMA_A initialized (base=0x41E00000)
DMA_B initialized (base=0x41E10000)
DMA_C initialized (base=0x41E20000)
=====
Hardware test input: digit 3
=====
Input =
  0  8  8  8  8  8  0  0
  0  0  0  0  0  8  0  0
  0  0  0  2  8  8  0  0
  0  2  8  8  8  2  0  0
  0  0  0  0  0  8  0  0
  0  0  0  0  0  8  0  0
  0  8  8  8  8  8  0  0
  0  0  0  0  0  0  0  0

Hardware predicted digit = 3
Hardware best normalized score x1000 = 7760
Hardware cycles = 4129787
Hardware latency = 41297 us
=====
Hardware test input: digit 6
=====

Hardware predicted digit = 6
Hardware best normalized score x1000 = 6965
Hardware cycles = 4129731
Hardware latency = 41297 us
Hardware-only tests completed.
```

Fig. 7: Hardware only latency test.

```

=====
Software-Only Latency Test
Tests only: digit 3 and digit 6
=====

=====
Software test input: digit 3
=====

Input =
 0  8  8  8  8  8  0  0
 0  0  0  0  0  8  0  0
 0  0  0  2  8  8  0  0
 0  2  8  8  8  2  0  0
 0  0  0  0  0  8  0  0
 0  0  0  0  0  8  0  0
 0  8  8  8  8  8  0  0
 0  0  0  0  0  0  0  0

Software predicted digit = 3
Software best normalized score x1000 = 7760
Software cycles = 12185140
Software latency = 121851 us

=====
Software test input: digit 6
=====

Input =
 0  6  8  8  6  0  0  0
 2  8  0  0  0  0  0  0
 6  8  8  8  6  0  0  0
 8  2  0  0  2  8  0  0
 8  2  0  0  2  8  0  0
 6  8  2  2  8  6  0  0
 0  6  8  8  6  0  0  0
 0  0  0  0  0  0  0  0

Software predicted digit = 6
Software best normalized score x1000 = 6965
Software cycles = 12185374
Software latency = 121853 us

Software-only tests completed.

```

Fig. 8: Software only latency test.

In both tests there are two input matrices of size 8x8, one representing a 3 and the other representing a 6. We can see that in the images above both the hardware and software tests recognize each input matrix, with both having the equal normalized score at the end (this is the score for which number best matches, more closely matched the higher the score).

The most interesting part of this test is the latency difference between the hardware only and software only tests. We can see with both hardware and software doing the same exact mathematical operations, the only difference being that the hardware run used our matrix multiplication IP is an improvement in latency of around 3x. Where using hardware only results in 0.041 seconds, compared to the software run where it results in 0.122 seconds. This displays just how faster our custom matrix multiplication IP is compared to using only the MicroBlaze V processor by itself.

Test #5: Multi-core/DDR Test

This test requires that the machine has a minimum of two cores capable of communicating with each other over DDR memory. The simplest way to do this is to have one core do arithmetic operations while the other core drives UART and waits for the result. An example of this would be to have core 1 load data into DDR memory for a 2x2 matrix multiplication. Core 2 will read the data from memory and perform matrix multiplication and return the result. Core 1 would have during that time sent data through UART and waited for the result before sending the result through UART.

This test must have both cores launch at the same time and not be performed in debug mode. In debug mode there are changes to the cache behavior which enable core communication to work in scenarios where it shouldn't.

```
core0 now loading data...
core1 now calculating...
[ 1 2
  3 4 ]
*
[ 4 9
  0 3 ]
=
[ 4 15
  12 39 ]
```

Fig. 9: Test results printed in minicom from UART connection to FPGA

Figure 9 shows the output of the multicore test showing successful communication over DDR memory between two cores running at the same time. The code for test 6 can be found in the test_5 directory.